

Binary Search Interview Guide

Reasoning First → Algorithm → Invariant → Complexity → Variations → Code Last

Study rule: Do not memorize binary-search code. First prove what lets you discard half the search space. Then define the invariant. Only after that write code.

PART I — THE BINARY SEARCH MENTAL MODEL

The key question is not “Where is mid?” but “**Why can I safely throw away one side?**” Binary search is valid only when some structure or monotonic property makes that decision reliable.

- Sorted array → target comparison identifies the impossible half.
- Boundary search → a monotonic true/false condition identifies a transition.
- Peak search → local slope guarantees a peak on one side.
- Rotated sorted array → sorted-half structure provides information.
- Answer-space search → feasibility is monotonic over candidate answers.

Loop invariant

A loop invariant is a statement that remains true before and after every iteration. In binary search it should describe **where the answer is guaranteed to remain**.

Peak example: “There is at least one peak inside [low, high].” Every update must preserve this guarantee.

Binary-search proof checklist

- Define the exact answer: any peak, strict peak, global maximum, first occurrence, etc.
- Identify the structure/monotonicity.
- Define the current candidate interval.
- Choose a safe mid.
- Prove why one half can be discarded.
- Preserve the answer in the remaining interval.
- Make the interval shrink.
- Prove termination.

PART II — SINGLE / UNIQUE ELEMENT

Single element in a sorted pair array

Brute force: scan/count or inspect neighbors → $O(n)$.

Observation: because the array is sorted and every other value occurs twice, equal values form adjacent pairs.

Optimal derivation: pair alignment has a parity pattern before the unique element and shifts after it. Therefore parity tells which side contains the unique element.

Invariant: the unique element remains inside the current interval; the discarded side has valid pair alignment and cannot contain it.

- LeetCode 540 — Single Element in a Sorted Array.
- Variation: unsorted array → binary-search structure disappears.
- Variation: elements occur three times → parity reasoning must be redesigned.

PART III — PEAK ELEMENT

Definition and constraints

A strict peak is greater than its neighbors. Boundary definitions depend on the problem. Always check whether adjacent duplicates are prohibited.

Brute force → better → optimal

Brute force: check every index and its neighbors → $O(n)$, $O(1)$ extra space.

Better observation: instead of checking every position, inspect the local direction around the middle.

Key observation: if $\text{nums}[\text{mid}] < \text{nums}[\text{mid}+1]$, the sequence is rising, so a peak must exist to the right. If $\text{nums}[\text{mid}] > \text{nums}[\text{mid}+1]$, a peak exists at mid or to the left.

Optimal algorithm: maintain an interval guaranteed to contain a peak. Rising → discard through mid. Falling → keep mid and discard the right side.

Invariant: $[\text{low}, \text{high}]$ always contains at least one peak.

Why high = mid? In a falling case, mid itself can be the peak. Using $\text{high} = \text{mid}-1$ could throw away the answer.

Why low < high? It guarantees $\text{mid} < \text{high}$, hence $\text{mid}+1$ is valid. When $\text{low} == \text{high}$, one candidate remains.

Duplicates: < vs <=

For LeetCode 162, adjacent elements are guaranteed different, so equality cannot occur. If duplicates are allowed, first decide whether a peak is strict ($>$) or non-strict ($\geq$). A plateau may have no strict peak but can contain non-strict peaks.

Do not change < to <= blindly. Equality changes the proof and sometimes changes the definition of a valid peak.

Multiple peaks vs maximum peak

Any peak: $O(\log n)$ is possible because the slope guarantees a peak in one half.

Largest peak in an arbitrary array: generally $O(n)$, because discarding half could discard a larger peak.

Maximum in a mountain/bitonic array: $O(\log n)$ because the increasing→decreasing structure identifies the turning point.

PART IV — MIN/MAX WITH STRUCTURE

Arbitrary global minimum / maximum

Brute force is already optimal: scan all values → $O(n)$. There is no general monotonic property allowing half the array to be discarded.

Interview lesson: do not force binary search onto an arbitrary unsorted array.

Mountain / bitonic array

Structure: increasing → one maximum → decreasing.

Brute force: scan maximum → $O(n)$.

Observation: rising at mid means maximum is right; falling means maximum is mid or left.

Invariant: the maximum remains inside $[\text{low}, \text{high}]$.

- LeetCode 852 — Peak Index in a Mountain Array.
- LeetCode 1095 — Find in Mountain Array.
- Variation: search a target in a bitonic array.

Rotated sorted array — minimum

Example structure: $[4, 5, 6, 7, 0, 1, 2]$.

Brute force: scan minimum → $O(n)$.

Key observation: because the array is a rotation of sorted order, endpoint/mid comparisons reveal useful sorted-half structure.

Invariant: the minimum remains in the current interval.

Warning: $\text{arr}[\text{low}] \leq \text{arr}[\text{mid}]$ does NOT imply the left half is sorted for an arbitrary array. That reasoning requires the rotated-sorted constraint.

- LeetCode 153 — Find Minimum in Rotated Sorted Array.
- LeetCode 154 — Find Minimum in Rotated Sorted Array II.
- Search in rotated sorted array; search with duplicates; find rotation count.

PART V — DUPLICATES

Why duplicates are difficult

If endpoint and midpoint values are equal, the comparison may not reveal which side contains the rotation point. In the worst case, the algorithm may need to shrink both ends, degrading toward $O(n)$.

General lesson: duplicates do not merely change an operator; they can destroy the information needed to eliminate half.

PART VI — BOUNDARY SEARCH

First / last occurrence, lower / upper bound

Brute force: scan to find the first/last matching position $\rightarrow O(n)$.

Observation: the desired position is a boundary in a monotonic condition.

Lower bound: first position with value $\geq$ target. Equality is not enough to stop because an earlier valid position may exist.

Upper bound: first position with value $>$ target.

Invariant: the desired boundary always remains inside the candidate interval.

- LeetCode 34 — Find First and Last Position.
- LeetCode 35 — Search Insert Position.
- Count occurrences = upper_bound – lower_bound.

PART VII — ANSWER-SPACE BINARY SEARCH

Brute force: try candidate answers one by one.

Observation: feasibility is often monotonic. Example: if capacity X works, every capacity larger than X also works.

Optimal: binary-search the answer range, not necessarily the array.

Invariant: the optimal answer remains inside the candidate answer interval.

- LeetCode 875 — Koko Eating Bananas.
- LeetCode 1011 — Capacity To Ship Packages Within D Days.
- LeetCode 1283 — Smallest Divisor Given a Threshold.
- LeetCode 410 — Split Array Largest Sum.
- LeetCode 1552 — Magnetic Force Between Two Balls.

PART VIII — BOUNDARY SAFETY

- Before $\text{mid}+1$, prove $\text{mid} < n-1$.
- Before $\text{mid}-1$, prove $\text{mid} > 0$.
- $\text{while}(\text{low} < \text{high})$ can make $\text{mid}+1$ safe in peak search.
- $\text{while}(\text{low} \leq \text{high})$ does not automatically make $\text{mid}+1$ safe.
- If using $\text{mid}-1$, protect index 0 or structure the search interval accordingly.

Always test: $n=1$, $n=2$, answer at index 0, answer at $n-1$, increasing, decreasing, multiple peaks, duplicate plateau, all equal, rotation by 0, rotation by 1.

PART IX — INTERVIEW VARIATIONS

Pattern	Core question	Typical complexity
Any peak	Does slope guarantee a peak on one side?	$O(\log n)$
Global max arbitrary	Can I safely discard half?	$O(n)$
Mountain max	Where does increasing become decreasing?	$O(\log n)$
Rotated min	Where is rotation point?	$O(\log n)$
Rotated + duplicates	Does equality hide the side?	Worst $O(n)$
Single unique	How does pair alignment shift?	$O(\log n)$
First occurrence	Can mid remain while searching left?	$O(\log n)$
Last occurrence	Can mid remain while searching right?	$O(\log n)$
Answer space	Is feasibility monotonic?	Often $O(n \log R)$

PART X — MUST-PRACTICE INTERVIEW QUESTIONS

- Find any peak in $O(\log n)$ and prove the discarded half cannot contain the only required answer.
- Explain why $\text{high} = \text{mid}$, not $\text{high} = \text{mid} - 1$, in peak search.
- Rewrite peak search using $\text{mid} - 1$ and explain the boundary changes.
- What happens if duplicates are allowed?
- Strict peak vs non-strict peak: what changes?
- Can you find the largest peak in an arbitrary array in $O(\log n)$? Prove your answer.
- Find the maximum in a mountain/bitonic array.
- Find minimum in a rotated sorted array.
- Why does $\text{arr}[\text{low}] \leq \text{arr}[\text{mid}]$ not imply sorted-left-half for an arbitrary array?
- What changes in a rotated sorted array with duplicates?
- Find the single element in a sorted array where every other element occurs twice.
- Find first and last occurrence of a target.
- Explain `lower_bound` and `upper_bound` using invariants.
- Explain answer-space binary search with a feasibility predicate.
- For every binary-search branch, state the invariant before and after the update.

PART XI — FINAL DECISION TREE

- Sorted? → classic/boundary binary search may apply.
- Rotated sorted? → reason about sorted halves and rotation point.
- Mountain/bitonic? → reason about slope and turning point.
- First/last/insert position? → boundary search.
- Capacity/speed/distance/days? → check monotonic feasibility and search the answer.
- Arbitrary unsorted global min/max? → usually $O(n)$.
- Duplicates? → re-check the proof; equality may remove the information needed to discard half.

PART XII — CODE: ONLY AFTER THE REASONING

These snippets are implementation references. The interview explanation should come first: definition → brute force → observation → optimal algorithm → invariant → boundary proof → complexity.

Peak — any peak

```

int low = 0, high = nums.size() - 1;
while (low < high) {
    int mid = low + (high - low) / 2;
    if (nums[mid] < nums[mid + 1]) low = mid + 1;
    else high = mid;
}
return low;

```

Lower bound

```

int low = 0, high = n;
while (low < high) {
    int mid = low + (high - low) / 2;
    if (arr[mid] >= target) high = mid;
    else low = mid + 1;
}
return low;

```

Upper bound

```

int low = 0, high = n;
while (low < high) {
    int mid = low + (high - low) / 2;
    if (arr[mid] > target) high = mid;
    else low = mid + 1;
}
return low;

```

Single element in sorted pairs

```

int low = 0, high = n - 1;
while (low < high) {
    int mid = low + (high - low) / 2;
    if (mid % 2) mid--;
    if (arr[mid] == arr[mid + 1]) low = mid + 2;
    else high = mid;
}
return arr[low];

```

Answer-space skeleton

```

long long low = minAnswer, high = maxAnswer;
while (low < high) {
    long long mid = low + (high - low) / 2;
    if (feasible(mid)) high = mid;
    else low = mid + 1;
}
return low;

```

FINAL INTERVIEW RULE

Never explain binary search as “take mid and throw away half.” Explain: “My invariant is X. Observation Y proves the answer lies in this half, so I can discard the other half while preserving X.”